UNIT 1

1) What is a socket?
A socket is a programming construct used for **communication between two processes** over a network. It is uniquely identified by an **IP address and port number**.

2) Analyze Client/Server architecture with respect to socket programming.

- **Server:** Creates a ServerSocket, waits for client requests, and provides services.
- **Client:** Creates a Socket and initiates communication with the server.
- Both exchange data using **input/output streams**.

3) Which type of communication is used for socket programming?
Socket programming supports:

- **Unicast** – one process to one process.
- **Multicast** – one process to multiple processes.
  It can use **TCP (connection-oriented)** or **UDP (connectionless)** communication.

4) Which package is used in Java for socket programming?
The **java.net package** is used for socket programming.

5) What are the two key classes used in Java to create client and server programs?

- **Socket** → used by the client.
- **ServerSocket** → used by the server.

6) Can we do multicasting using sockets? How?
Yes. Multicasting can be done using **UDP datagrams**, where one sender sends data to multiple receivers belonging to a multicast group.

7) Which Java API is used to implement multicasting?
The **MulticastSocket** class from the **java.net package** is used for multicasting.

8) Compare socket programming using TCP/IP and UDP.

TCP	**UDP**
Connection-oriented	Connectionless
Reliable	Not guaranteed reliable
Data received in order	Packets may arrive out of order
Uses Socket / ServerSocket	Uses DatagramSocket / DatagramPacket
Suitable for reliable communication	Suitable for faster communication

9) What is a port? Which are reserved ports? Which ports are available for socket communication?
A **port** is a number used to identify a communication endpoint on a machine. Port numbers are **16-bit**. Traditionally:

- **0–1023:** Well-known/reserved ports.
- **1024–65535:** Available for user-defined socket applications.
  The provided lab manual specifically states that user-defined processes should use ports **greater than 1024**.

10) Applications of socket programming + one real-life application.
Applications include:

- Chat applications
- Online multiplayer games
- Web communication
- File transfer
- Client-server applications
- Real-time messaging

Real-life example – Chat application:
A chat server listens on a specific port using ServerSocket. Multiple clients connect using Socket, and messages are exchanged through input/output streams. This allows users to communicate in real time.

UNIT 2

1) What is Remote Procedure Call (RPC)? Explain with architecture.
RPC is a mechanism that allows a **client to call a procedure located on a remote server** as if it were a local procedure. The client sends parameters and waits for the result.

Architecture:

Client → Client Stub → Client OS → Network → Server OS

↓

Server Stub → Server

↓

Result ←————

Steps: Client calls stub → stub packs parameters → message travels to server → server stub unpacks → server executes → result is packed and sent back.

---

2) What is Stub and Skeleton in RPC?

- **Stub:** Client-side code that receives the procedure call, **packs (marshals) parameters** and sends them to the server.
- **Skeleton:** Server-side code that **unpacks (unmarshals) parameters**, calls the actual server procedure, and returns the result.

---

3) How is parameter passing done in RPC?
Parameters are **marshalled** into a message by the client stub and sent over the network. The server stub **unmarshals** them and passes them to the server procedure. The result is similarly packed and returned.

---

4) What is Portmapper? Explain its use.
Portmapper maps an RPC **program number/version to the port number** on which the RPC service is running.

Client

|

| Request RPC service

↓

Portmapper (Server)

|

| Returns port number

↓

RPC Server

|

| Performs procedure

↓

Client

It helps the client locate the correct port of an RPC service.

---

5) Explain steps in writing a client and server in DCE RPC.
Basic steps are:

1. Define the interface using **IDL**.
2. Generate client/server code from the IDL.
3. Compile the generated code.
4. Implement the server procedure.

5. Implement the client program.
6. Start the server.
7. Run the client and perform the RPC call.

The provided lab follows a similar **SUN RPC/rpcgen** workflow: create IDL → generate code → test client/server → implement server work → make client functional → compile and execute.

---

6) What is Portmapper? Explain its use.
Portmapper is a service that helps an RPC client **find the port number associated with a particular RPC service**. It avoids the need for the client to know the server's RPC port beforehand.

---

7) Which necessary files are generated by rpcgen?
Common files are:

- .h → Header/interface definitions
- _clnt.c → Client stub
- _svc.c → Server stub/skeleton
- _xdr.c → XDR data conversion code
- Client and server source files

For example, the assignment generates files such as **add_client.c and add_server.c**.

---

8) What are the different data types used in SUNRPC?
Common SUNRPC/XDR data types include:

- int
- unsigned int
- short
- long
- float
- double
- char
- string
- array
- struct
- enum
- bool

---

9) Give at least two applications of RPC.

1. **Distributed database systems** – Client requests database operations from a remote server.
2. **File/Network services** – A client can request files or services from a remote machine.

Other examples include remote administration and distributed computing applications.

---

10) RPC code to calculate sum of array elements

IDL (sum.x):

```
program SUM_PROG {
   version SUM_VERS {
      int SUM(int arr<>) = 1;
   } = 1;
} = 0x20000001;
```

Server:

```
int *sum_1_svc(int arr[], struct svc_req *req)
{
   static int result;
   int i, n = sizeof(arr) / sizeof(arr[0]);

   result = 0;
   for(i = 0; i < n; i++)
      result += arr[i];

   return &result;
}
```

Client:

```
int arr[5] = {10, 20, 30, 40, 50};

int *result;
result = sum_1(arr, clnt);
```

printf("Sum = %d\n", *result);

The general RPC workflow in the assignment is that the client passes parameters to the remote procedure, the server performs the operation, and the result is returned to the client.

UNIT 3

1) What is a Distributed Object? Explain Distributed Object Model.

A **distributed object** is an object whose methods can be invoked by a client located in another process or machine.

Distributed Object Model:

Client JVM Server JVM

| |

Client → Proxy/Stub — Network —→ Remote Object

| |

←————————— Result ——————————|

Java RMI allows an object in one JVM to invoke methods of an object in another JVM.

---

2) Difference between Local Object and Remote Object

Local Object	**Remote Object**
Exists in the same JVM/process	Exists in another JVM/process
Method call is local	Method call occurs over network
No network communication	Network communication required
Faster	Relatively slower
Normal Java object	Implements a remote interface

---

3) What is Serialization? Write serialized form for Employee.

Serialization is the process of converting an object's state into a byte stream so that it can be transmitted or stored. In RMI, parameters are serialized before being sent to the remote JVM.

The class should implement Serializable:

import java.io.Serializable;

```
public class Employee implements Serializable {
   private String name;
   private String Address;
   private String Deptname;
   private int Empid;

   public Employee(String s, String a, String dname, int id) {
      name = s;
      Address = a;
      Deptname = dname;
      Empid = id;
   }
}
```

---

4) Election Interface – Input and Output Parameters

Vote(candidate, voterNumber):

- candidate → **Input**
- voterNumber → **Input**

The client supplies both values to the server.

result(candidate, votes):

- candidate → **Output**
- votes → **Output**

The server supplies both values to the client.

---

5) Use of Reflection Interface in Java. How to construct client proxy?

Reflection allows a Java program to inspect classes, interfaces, methods, and objects at runtime.

Example:

```
Class<?> c = Class.forName("Election");

for (java.lang.reflect.Method m : c.getMethods()) {
   System.out.println(m.getName());
```

}

A proxy can be created dynamically using:

```
Election proxy = (Election) java.lang.reflect.Proxy.newProxyInstance(
    Election.class.getClassLoader(),
    new Class<?>[]{Election.class},
    handler
);
```

The proxy forwards method calls to the remote service.

---

6) How is parameter passing done in Java RMI?

Java RMI uses **serialization** for parameters.

- **Primitive values** → passed by value.
- **Serializable objects** → object state is serialized and passed by value.
- **Remote objects** → remote reference/stub is passed.

The RMI stub marshals (serializes) parameters before sending them to the remote JVM.

---

7) What is Dynamic Class Loading in Java RMI?

Dynamic class loading means RMI can **download and load a required class at runtime** if the client does not already have that class.

Example: A server updates a class and places it on a web server; the client can automatically fetch the new class when required.

---

8) Role of Security Manager in Java RMI

The **Security Manager** controls what remote code is allowed to do on a machine.

It helps protect resources by restricting operations such as:

- File access
- Network access
- System resource access

The assignment notes the use of RMISecurityManager to protect server resources during remote communication.

---

9) Static vs Dynamic Remote Method Invocation

Static Invocation	Dynamic Invocation
Interface/stub information is known beforehand	Method information can be determined at runtime
More compile-time dependent	More runtime flexible
Requires predefined interfaces/classes	Can dynamically load required classes
Faster and simpler	More flexible

---

10) Java Code to Implement Election Service using RMI

Election.java

```
import java.rmi.*;

public interface Election extends Remote {
   void vote(String candidate, int voterNo) throws RemoteException;
   int result(String candidate) throws RemoteException;
}
```

ElectionImpl.java

```
import java.rmi.*;
import java.rmi.server.*;

public class ElectionImpl extends UnicastRemoteObject
      implements Election {

   private int votes = 0;

   ElectionImpl() throws RemoteException {}

   public void vote(String candidate, int voterNo)
         throws RemoteException {
      votes++;
   }
```

```
    public int result(String candidate)
            throws RemoteException {
        return votes;
    }
}
```

Server.java

```
import java.rmi.*;

public class Server {
    public static void main(String[] args) throws Exception {
        ElectionImpl obj = new ElectionImpl();
        Naming.rebind("Election", obj);
        System.out.println("Election Server Started");
    }
}
```

Client.java

```
import java.rmi.*;

public class Client {
    public static void main(String[] args) throws Exception {

        Election e = (Election)
            Naming.lookup("rmi://localhost/Election");

        e.vote("CandidateA", 101);
        e.vote("CandidateA", 102);

        System.out.println("Votes = " +
            e.result("CandidateA"));
    }
}
```

Output:

Election Server Started

Votes = 2

The RMI workflow used here follows the assignment: define a remote interface → implement the remote object → bind it to the RMI registry → client performs Naming.lookup() → invoke the remote method.

The assignment also specifies **RMI Registry port 1099 by default**.